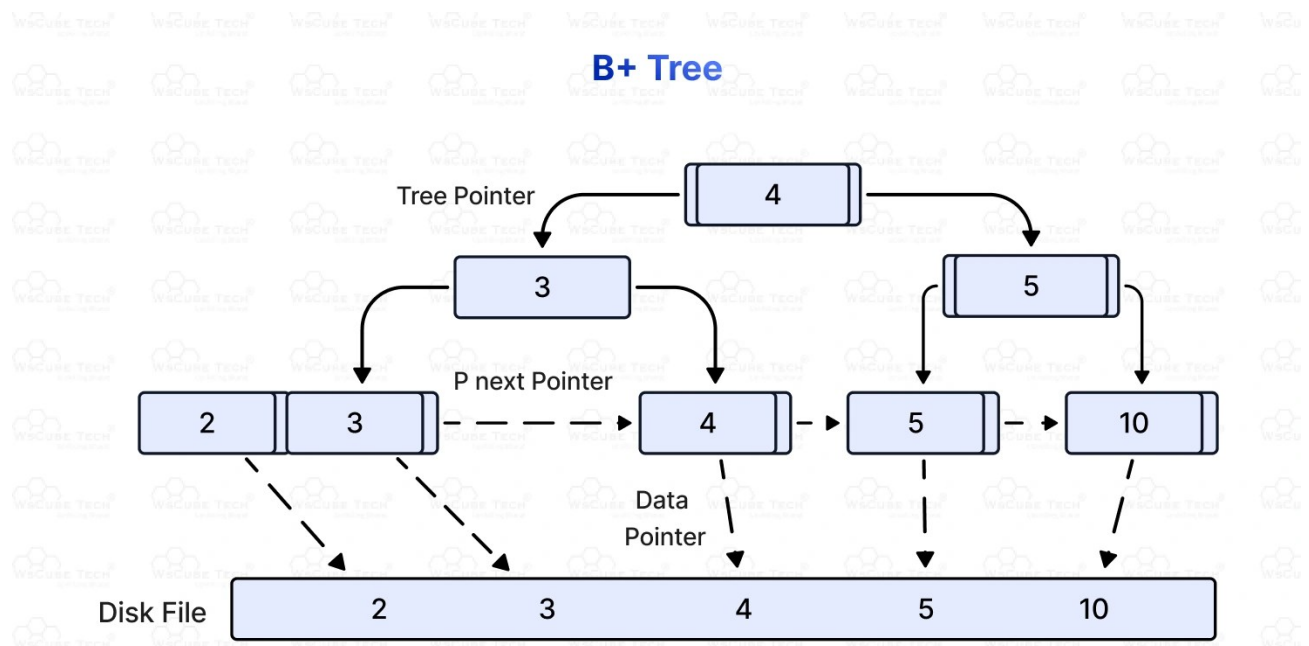


B+ Tree

A B+ Tree is a type of data structure used in computer science to store and organize data. It is a special kind of tree where all the values (data) are stored in the leaf nodes, and the internal nodes are used only for indexing (like a table of contents). This structure helps in quickly finding, inserting, and deleting data. In a B Plus Tree, the leaf nodes are linked together like a linked list, which makes it easy to find a range of values quickly.

For example, if you want to find all the values between 10 and 20, you can quickly go to the starting point and then follow the links to get all the values in that range.



Properties of B+ Tree

- **Balanced Tree Structure:** B+ Trees are balanced, meaning all leaf nodes are at the same level.
- **Leaf Nodes Store All Values:** All actual data values are stored in the leaf nodes.
- **Internal Nodes for Indexing:** Internal nodes contain only keys and pointers to child nodes.
- **Linked List of Leaf Nodes:** Leaf nodes are linked together in a linked list.
- **Order of the Tree:** The order (m) of a B+ Tree defines the maximum number of children each internal node can have.

- **Minimum Degree:** The minimum degree (t) of a B+ Tree is defined as $\lceil m/2 \rceil$.
- **Efficient Disk Read/Write:** B+ Trees are designed to minimize disk I/O operations.

Nodes in B+ Tree

In a B+ Tree, nodes are of two main types: internal nodes and leaf nodes.

- **Internal Nodes**

Internal nodes are used for indexing and routing purposes within the tree. They do not store actual data values but rather keys that help in navigating the tree.

- **Leaf Nodes**

Leaf nodes store the actual data values. All the data in a B+ Tree is contained within the leaf nodes.

Order of B+ Tree

The order of a B+ Tree, often denoted as m , is a fundamental property that determines the tree's structure and balance. The order of a B+ Tree specifies the maximum number of children that any node in the tree can have. Here's a detailed explanation of what the order of a B+ Tree means and its implications:

Order m :

The order of a B+ Tree is the maximum number of children that any internal node can have.

For a B+ Tree of order m :

- Each internal node can have at most m children.
- Each internal node (except the root) must have at least $\lceil m/2 \rceil$ children.
- Each internal node must have between $\lceil m/2 \rceil - 1$ and $m - 1$ keys.
- Each leaf node must have between $\lceil m/2 \rceil$ and m values.

Operations on B+ Trees

B+ Trees support various operations such as insertion, deletion, and searching. These operations ensure that the tree remains balanced and efficient for data retrieval and modification.

1. Insertion

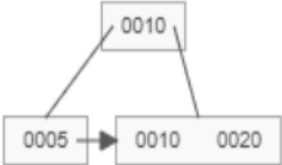
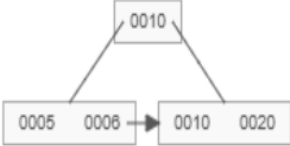
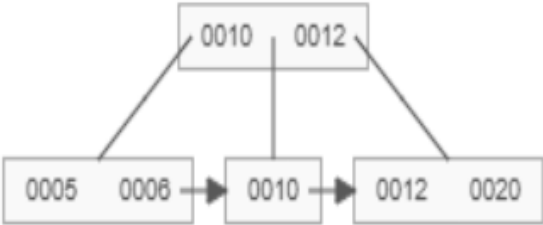
Algorithm:

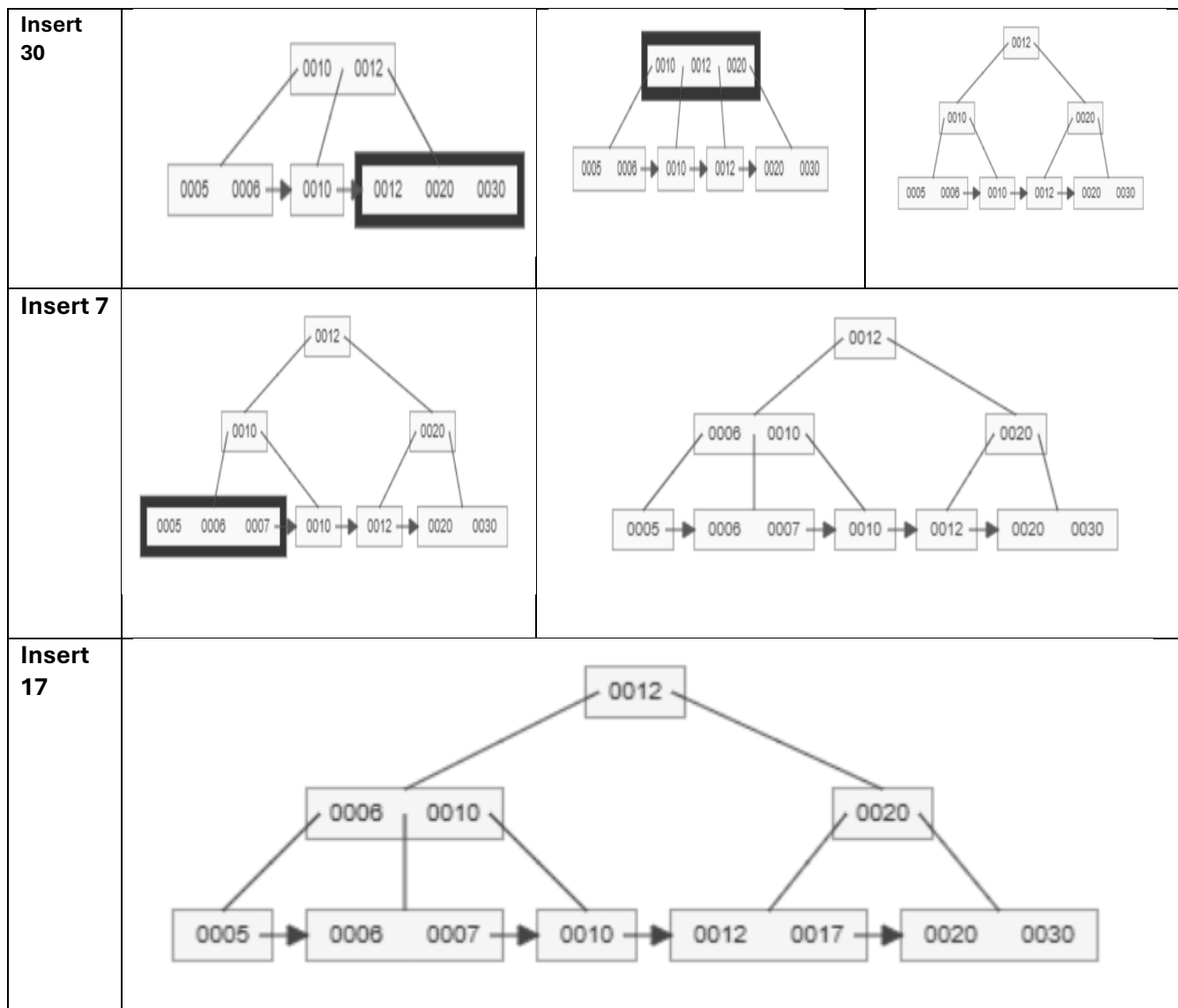
1. Start from the root and find the appropriate leaf node where the key should be inserted.
2. Insert the key in the leaf node in sorted order.
3. Split the Node (if necessary):
 - If the leaf node has more than $m-1$ keys after insertion, split the node into two.

- Promote the middle key to the parent node.
 - If the parent node also overflows, repeat the splitting process up to the root.
4. Ensure that leaf nodes are linked correctly after any split.

Example: Insert keys 10, 20, 5, 6, 12, 30, 7, 17 in a B+ Tree of order 4.

[10]

Insert 10:			
Insert 20:			
Insert 5			
Insert 6			
Insert 12			



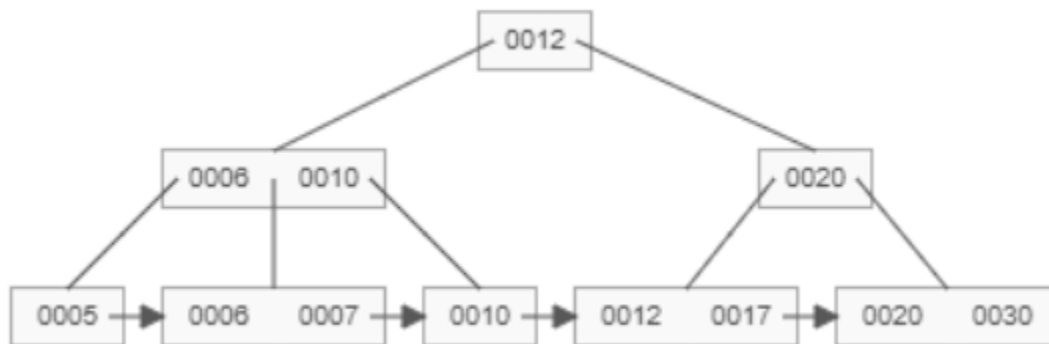
2. Deletion

Algorithm:

1. Start from the root and find the leaf node containing the key.
2. Remove the key from the leaf node.
3. Merge or Redistribute Nodes (if necessary):
 - If the leaf node has fewer than $\lceil m/2 \rceil$ keys after deletion, merge or redistribute nodes.
 - If merging, remove the corresponding key from the parent and merge nodes.
 - If the parent node underflows, repeat the process up to the root.

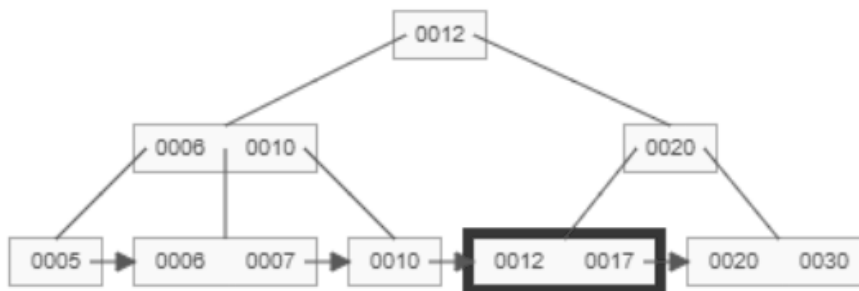
Example:

Delete key 12 from the B+ Tree

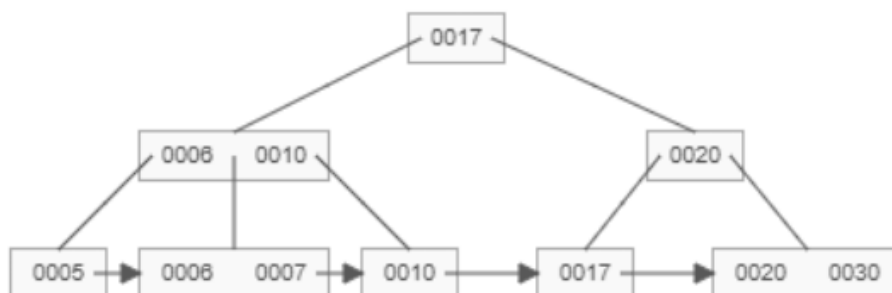


Solution:

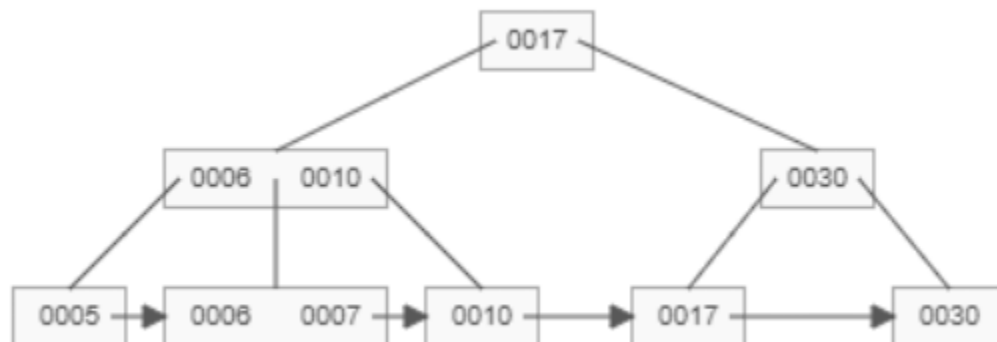
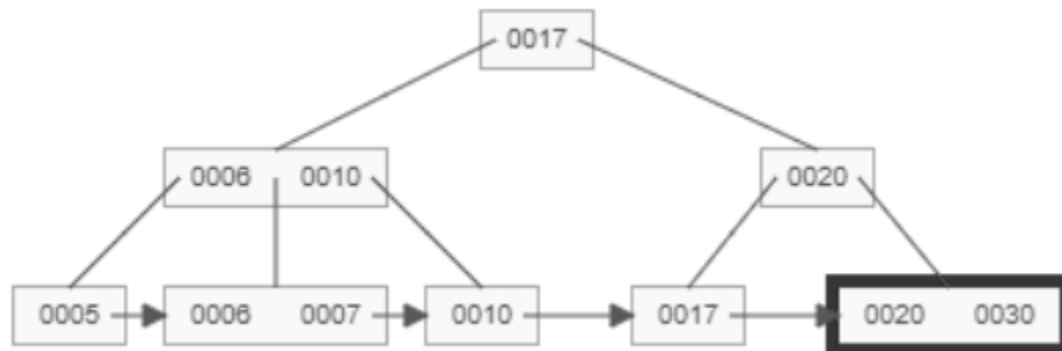
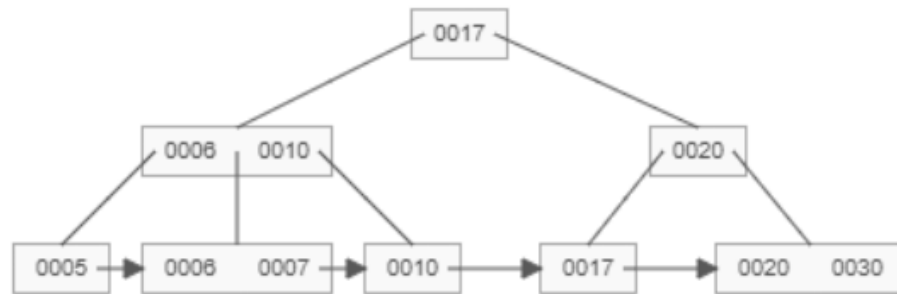
Step 1 : Find 12:



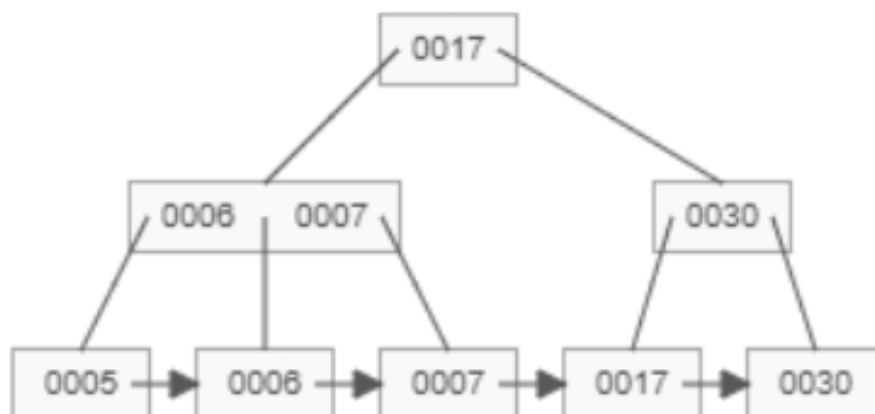
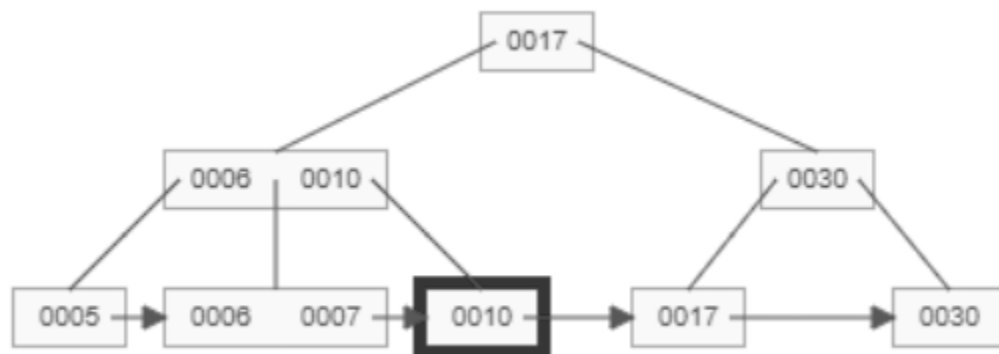
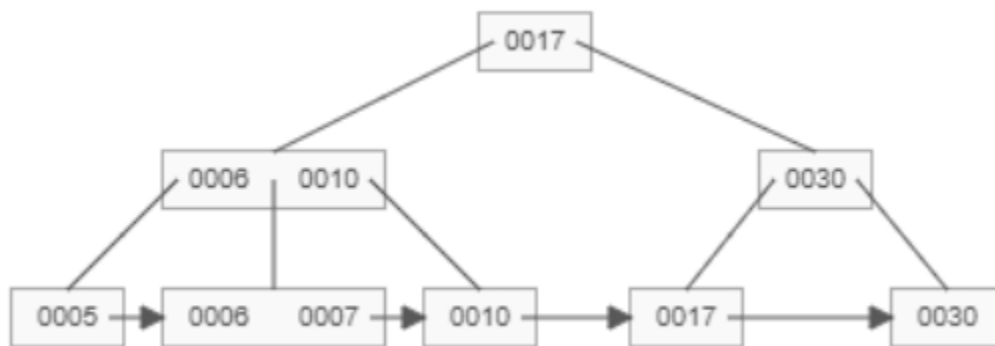
Step 2 :



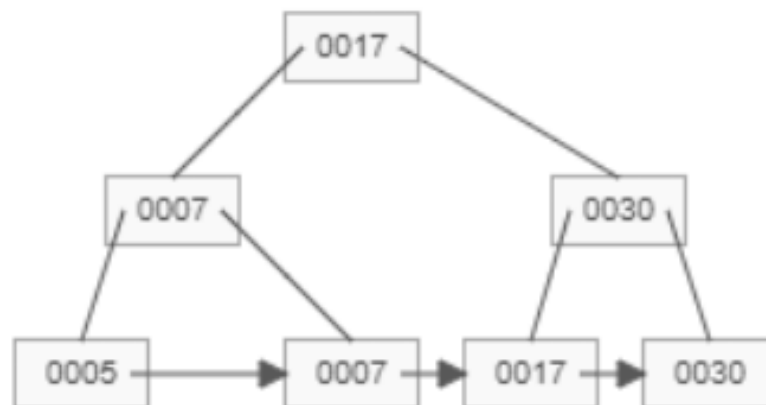
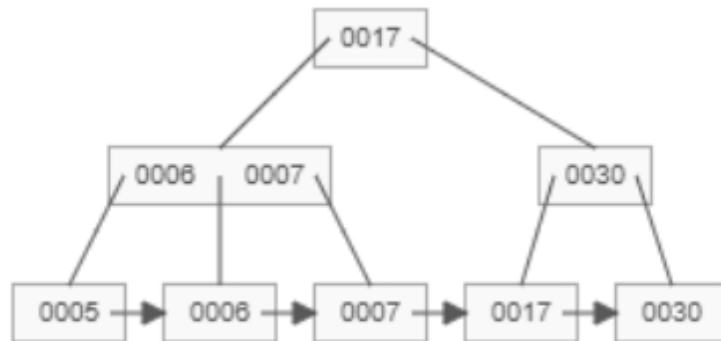
Delete 20



Delete 10:



Delete 6



3. Searching

Algorithm:

1. Begin the search from the root node.
2. Compare the key with the keys in the current node:
 - If the key is less than a key in the node, follow the corresponding child pointer.
 - If the key is greater, move to the next key or child pointer.
3. Continue this process until you reach the leaf node.
4. Look for the key in the leaf node.

Question 1 :

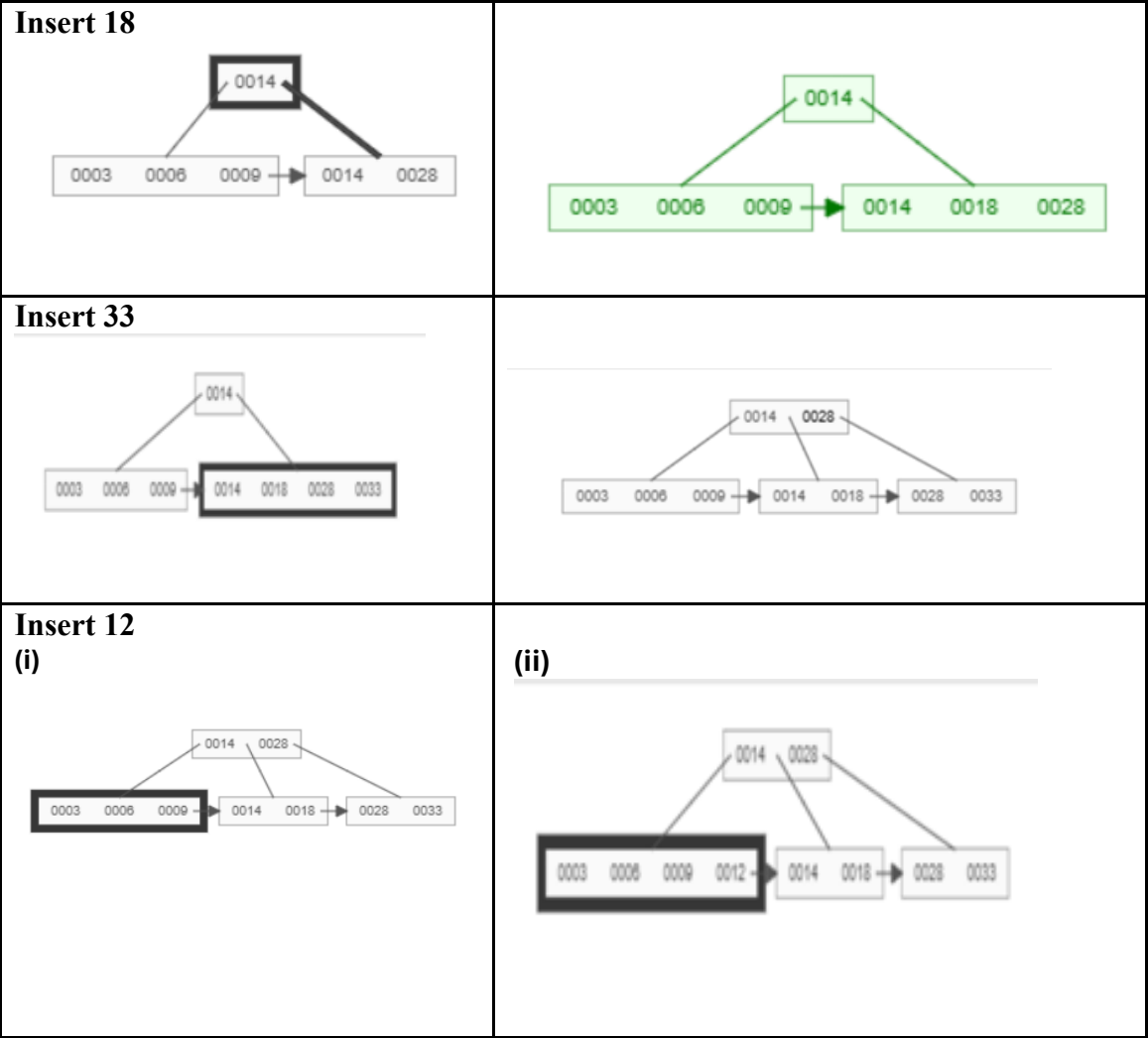
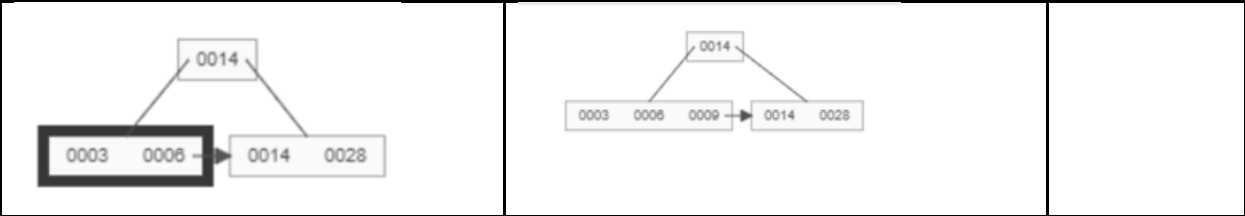
Construct a B+ Tree of order 5 by inserting the keys 6, 14, 28, 3, 9, 18, 33, 12, 21, 26 in the given order. Show leaf and internal node splitting, key promotion, and leaf-level links after each insertion. After construction, insert 24 and 36, then delete 9 and 18. Show all redistribution or merging steps during deletion.

Solution:

B+ Tree Construction

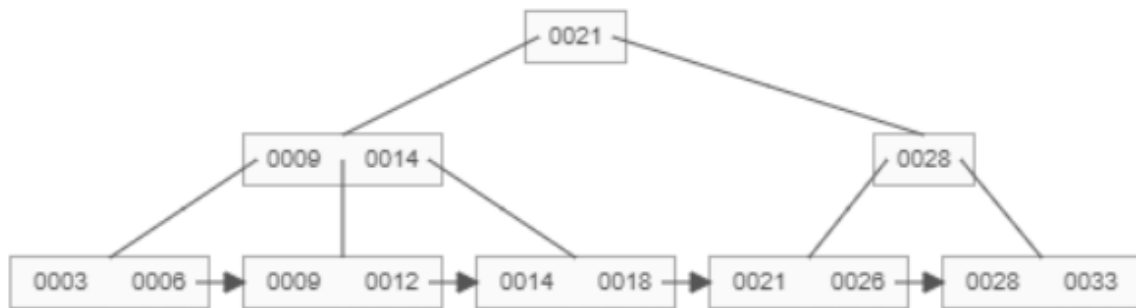
Insertion operation {6, 14, 28, 3, 9, 18, 33, 12, 21, 26}

Insert 6 		
Insert 14 		
Insert 28 		
Insert 3 		
Insert 9		

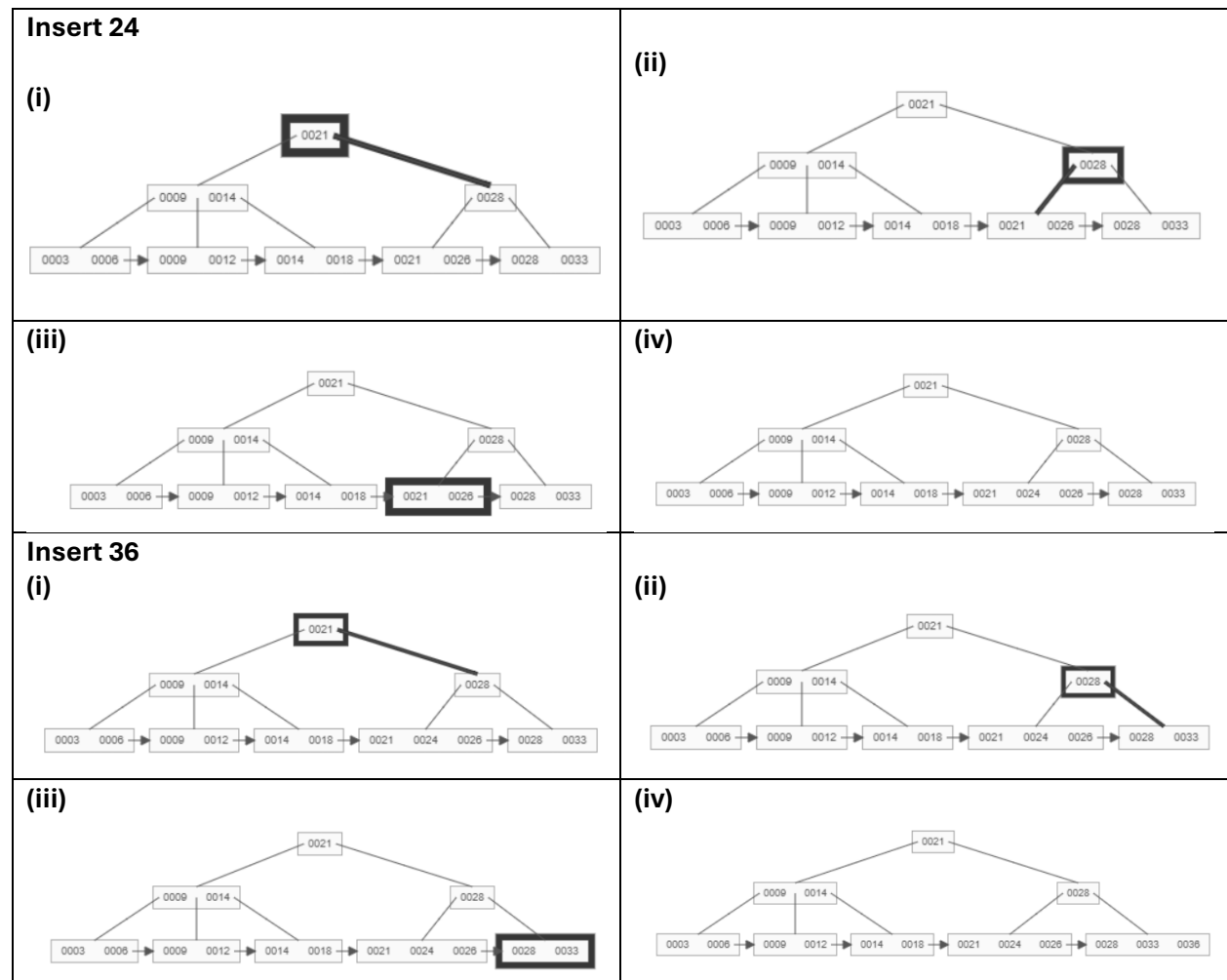


	(iii)
Insert 21 (i)	(ii)
Insert 26 (i)	(ii)
(iii)	(iv)
(v)	(v)

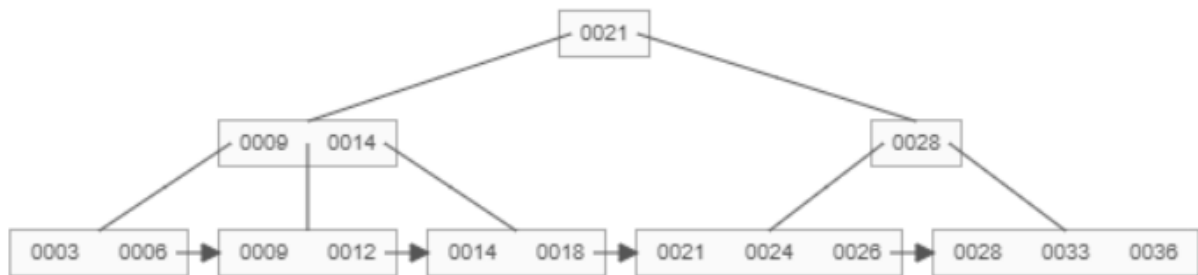
Finally tree



Insertion 24 and 36



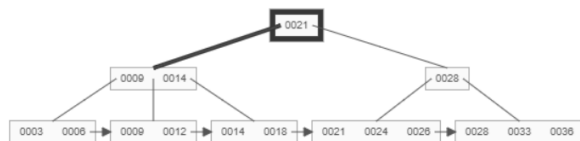
After inserting 24 and 36



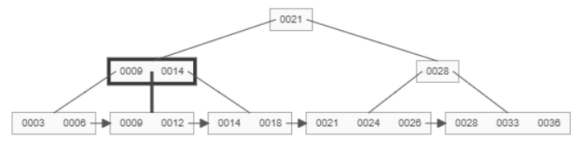
Deletion of 9 and 18

Delete 9

(i)



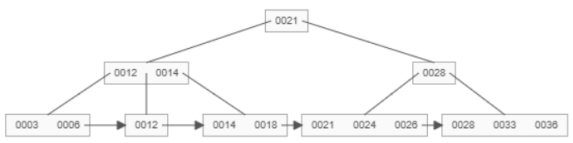
(ii)



(iii)

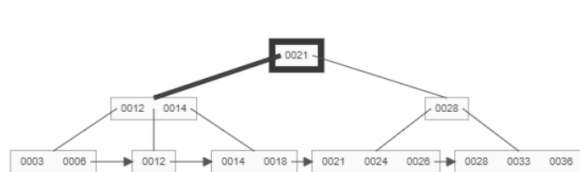


(iv)

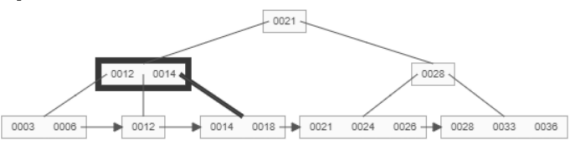


Delete 18

(i)



(ii)



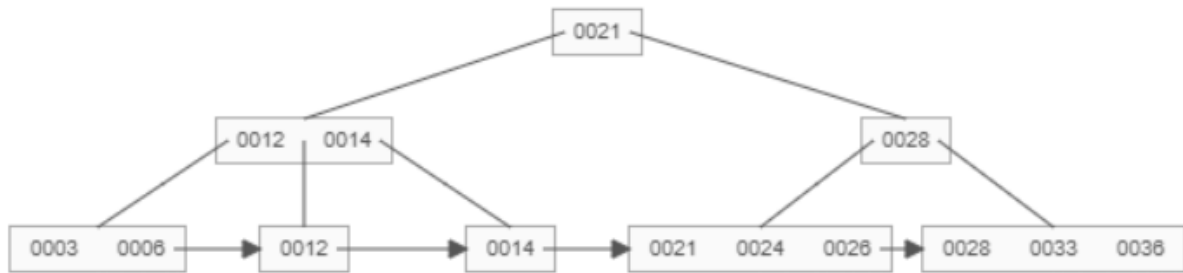
(iii)



(iv)



Final tree



Question 2 :

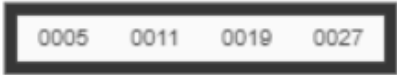
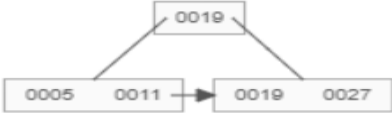
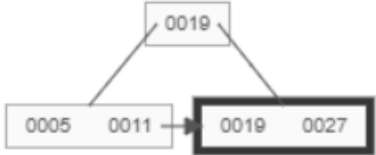
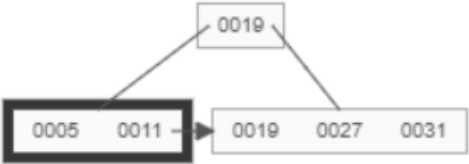
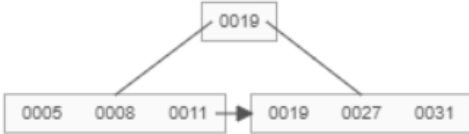
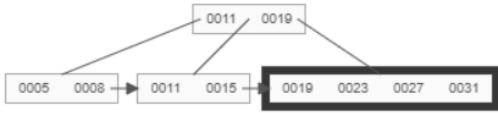
Construct a B+ Tree of order 5 by inserting the keys 11, 27, 5, 19, 31, 8, 15, 23, 35, 29 in the given order. Illustrate splitting of leaf and internal nodes, promoted separator keys, and linked leaves. After forming the tree, insert 17 and 38, then delete 5 and 27. Show all deletion steps using redistribution or merging.

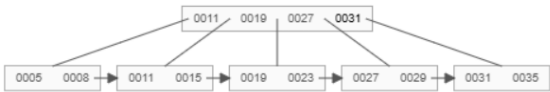
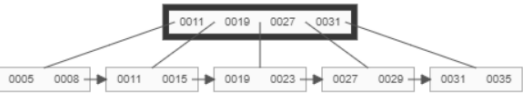
Solution:

B+ Tree Construction

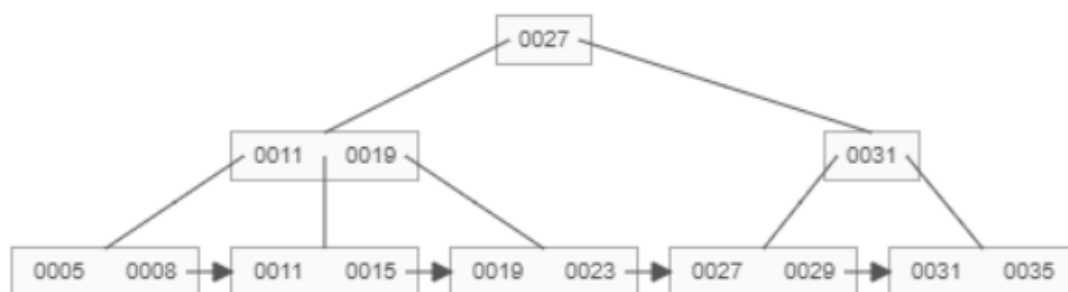
Insertion operation {11, 27, 5, 19, 31, 8, 15, 23, 35, 29}

Insert 11 0011		
Insert 27 0011 0027		
Insert 5 0005 0011 0027		

Insert 19 	
Insert 31 	
Insert 8 	
Insert 15 	
Insert 23 	

Insert 35 	
Insert 29 (i) 	(ii) 
(iii) 	

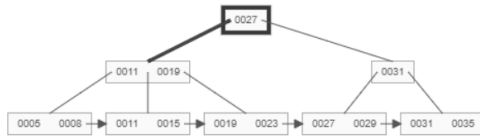
Finally tree after insertion



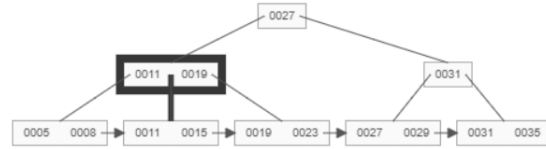
(i) Inserting 17 and 38

Insert 17

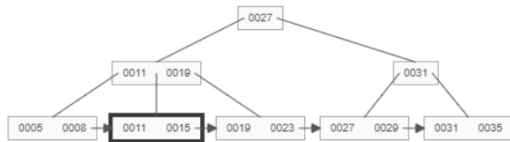
(i)



(ii)



(iii)



(iv)



Insert 38

(i)



(ii)



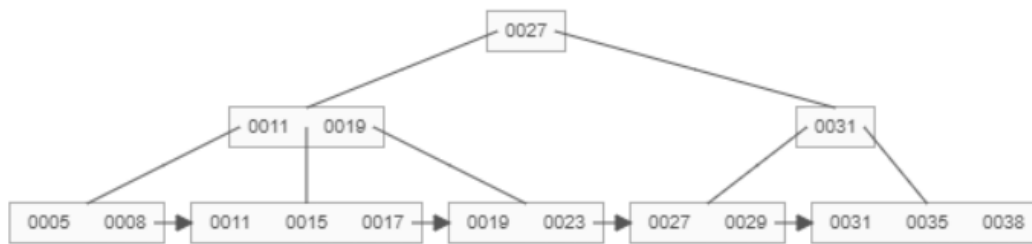
(iii)



(iv)



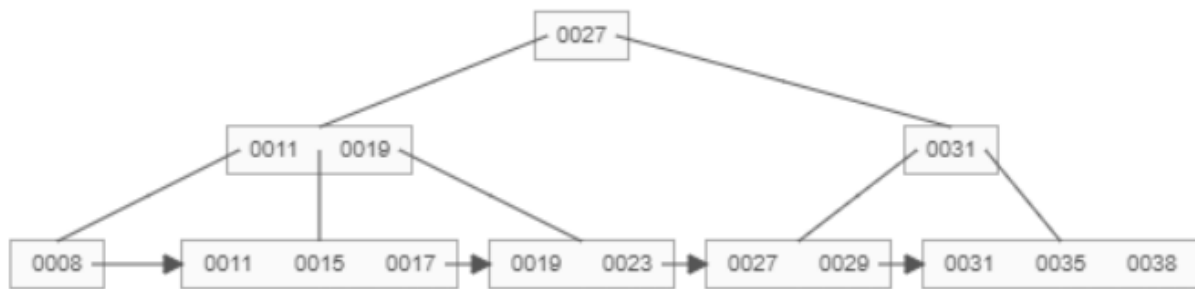
Final tree after inserting



Deleting 5 and 27

Delete 5 (i)	(ii)
(iii)	(iv)

Final tree after deleting 5

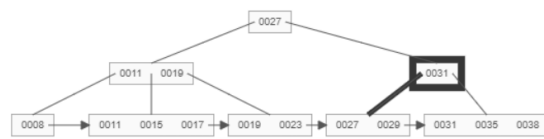


Delete 27

(i)



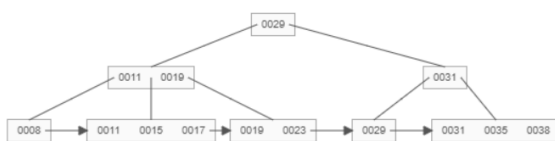
(ii)



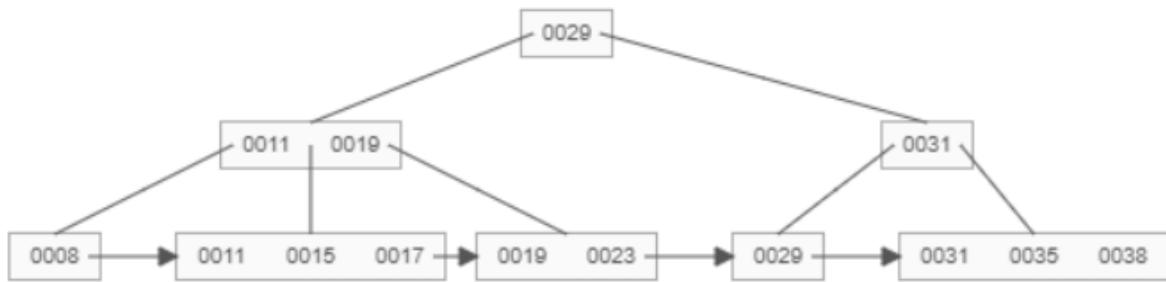
(iii)



(iv)



Final tree



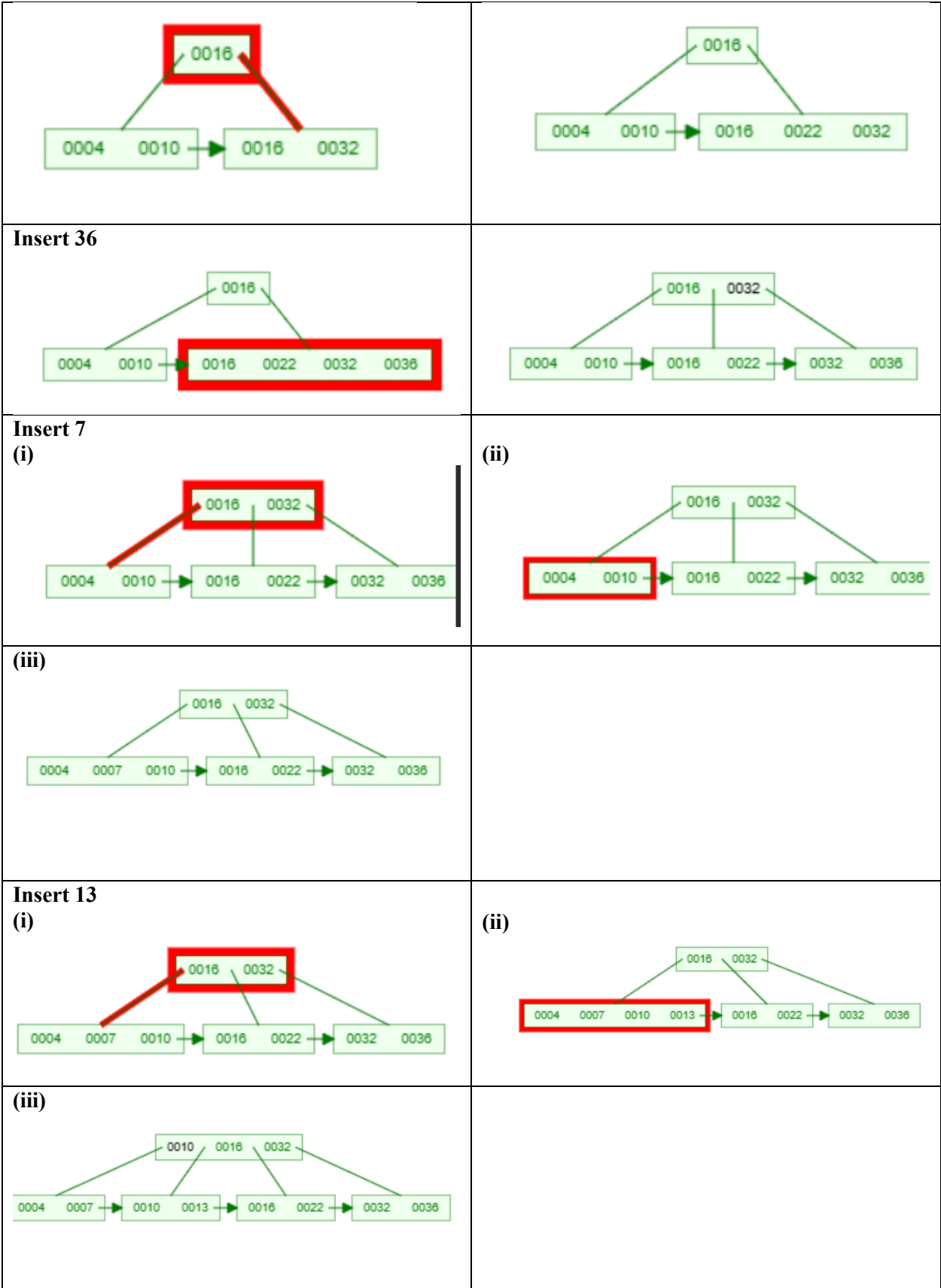
Question 3

Construct a B+ Tree of order 5 by inserting the keys 4, 16, 32, 10, 22, 36, 7, 13, 19, 28 in the given order. Show node splitting, key promotion, and maintenance of leaf-node pointers. After construction, insert 25 and 40, then delete 10 and 32. Explain redistribution and merging during deletion.

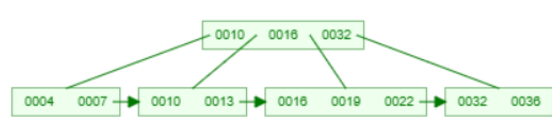
B+ Tree Construction

Insertion operation {4, 16, 32, 10, 22, 36, 7, 13, 19, 28}

Insert 4 0004	
Insert 16 0004 0016	
Insert 32 0004 0016 0032	
Insert 10 0004 0010 0016 0032	
Insert 22	

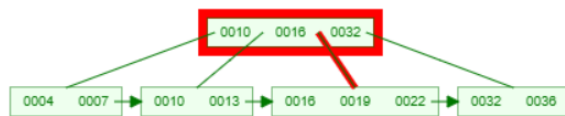


Insert 19



Insert 28

(i)



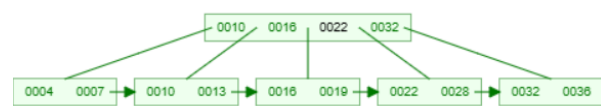
(ii)



(iii)



(iv)



(v)



(vi)



Finall tree after insertion



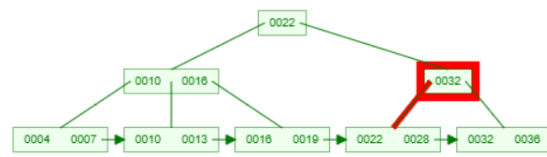
inserting 25 and 40

Insert 25

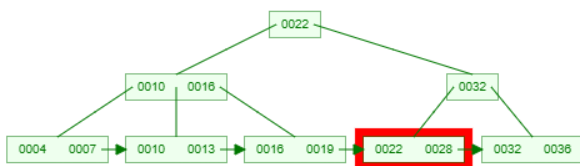
(i)



(ii)



(iii)



(iv)

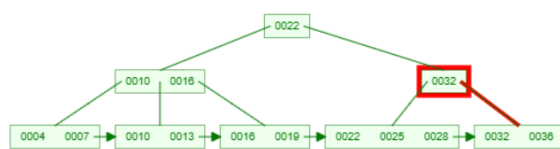


Insert 40

(i)

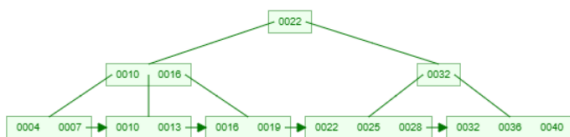


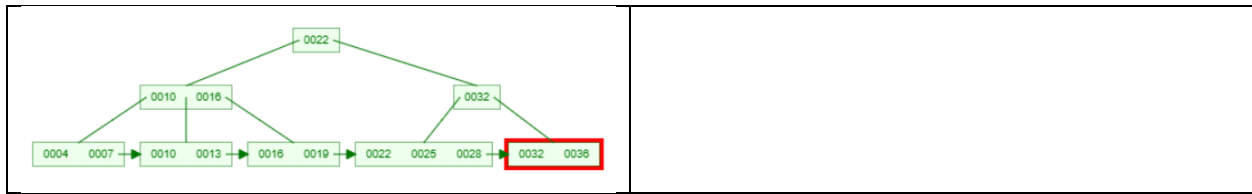
(ii)



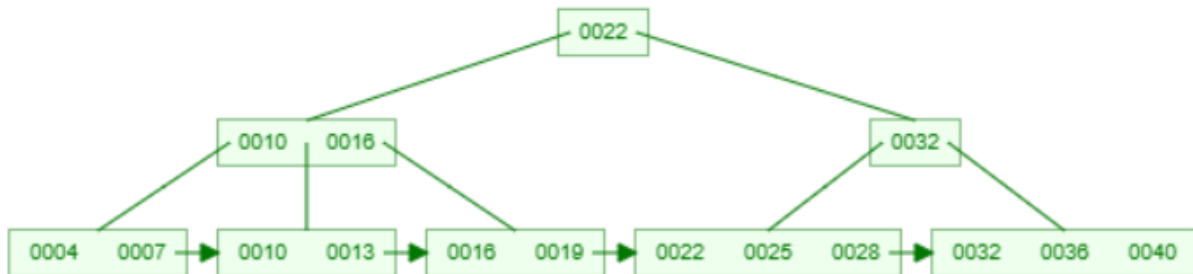
(iii)

(iv)

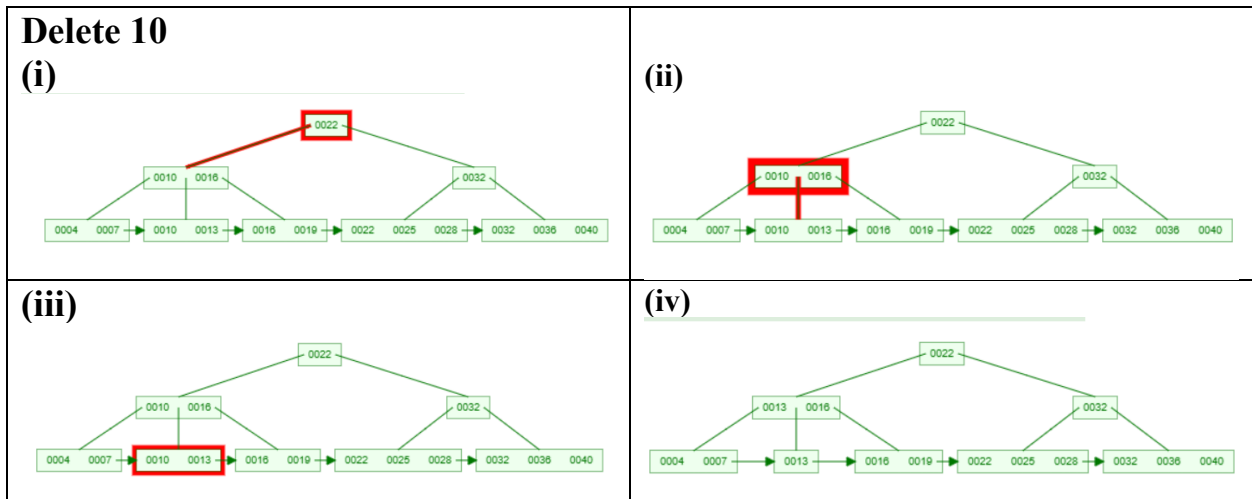




Final tree after inserting 25 and 40

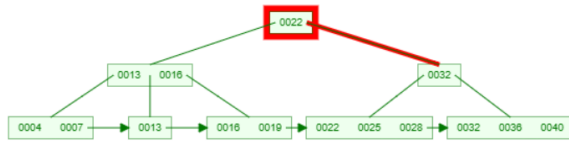


Deletion of 10 and 32

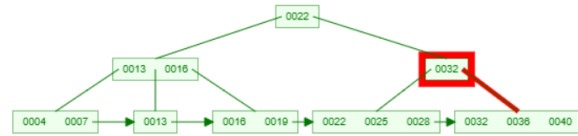


Delete 32

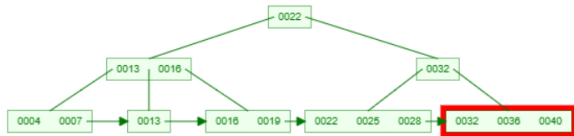
(i)



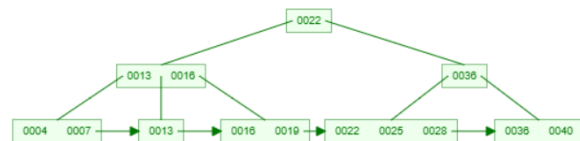
(ii)



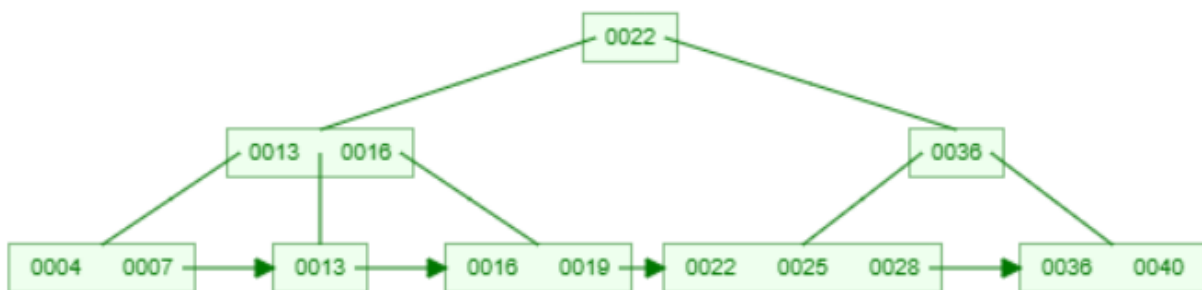
(iii)



(iv)



Final tree



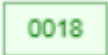
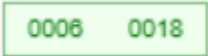
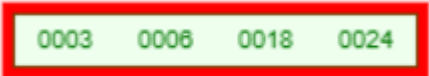
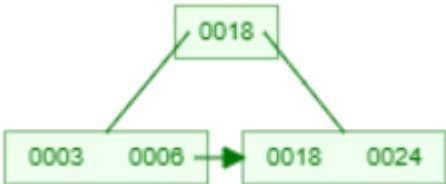
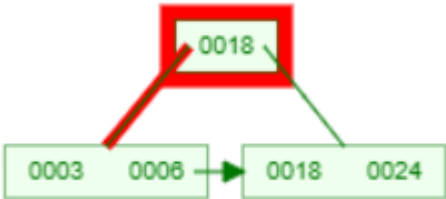
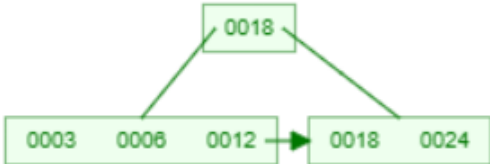
Question 4

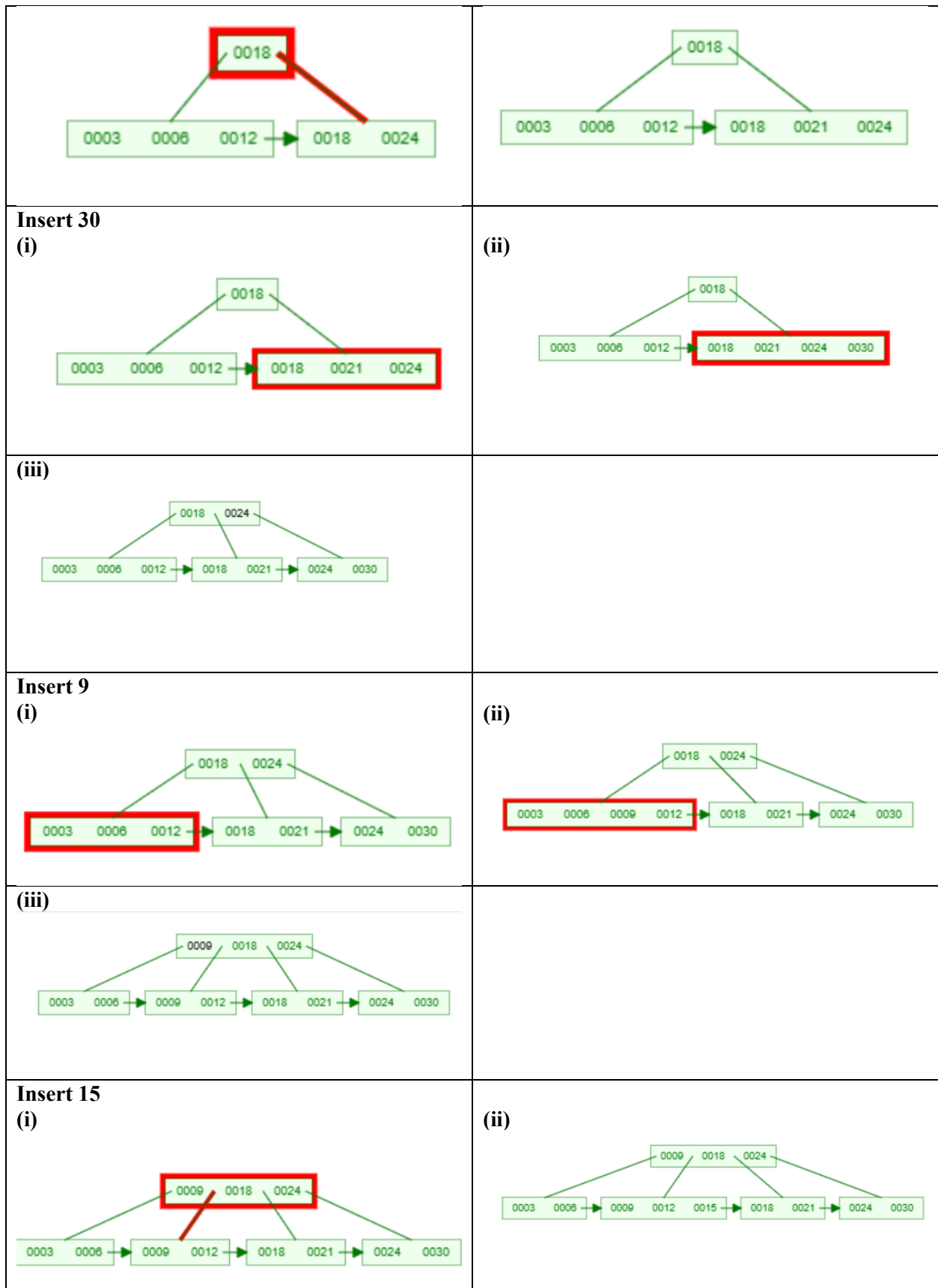
Construct a B+ Tree of order 5 by inserting the keys 18, 6, 24, 3, 12, 21, 30, 9, 15, 27 in the given order. Show the B+ Tree after each insertion with splits, promotions, and leaf-level linking. After obtaining the

final tree, insert 33 and 36, then delete 6 and 24. Show all deletion steps involving redistribution or node merging.

B+ Tree Construction

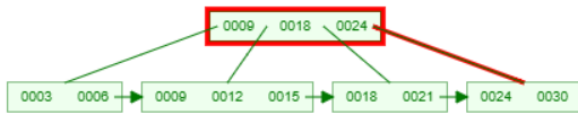
Insertion operation {18, 6, 24, 3, 12, 21, 30, 9, 15, 27}

Insert 18 	
Insert 6 	
Insert 24 	
Insert 3 	
Insert 12 	
Insert 21	

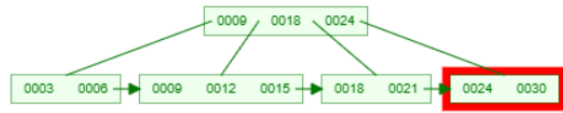


Insert 27

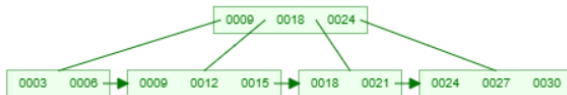
(i)



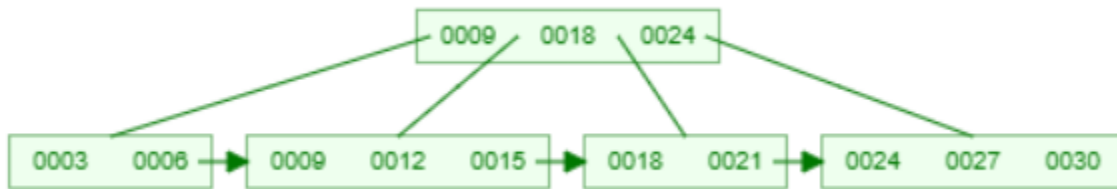
(ii)



(iii)



Final tree after insertion



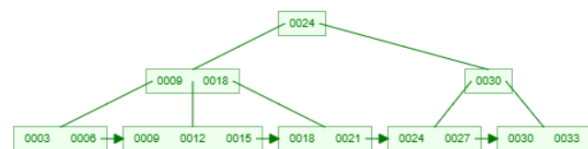
inserting 33 and 36

Insert 33

(i)



(ii)



Insert 36

(i)



(ii)



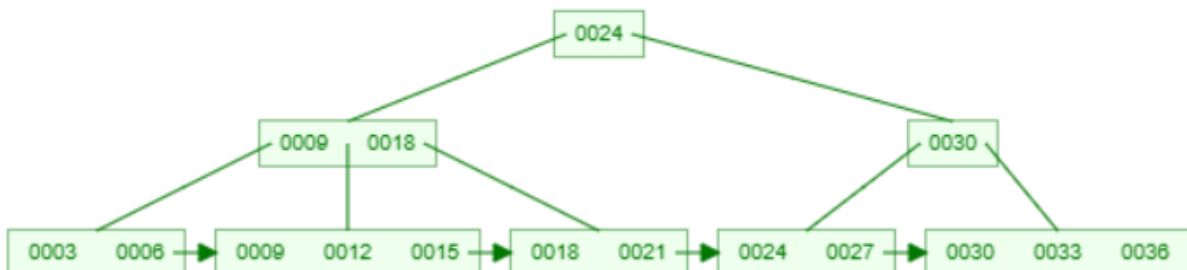
(iii)



(iv)



Final tree after inserting 33 and 36



Deletion of 6 and 24

Delete 6

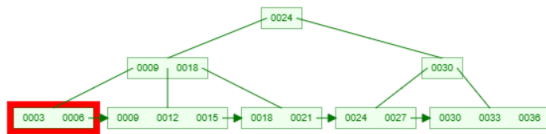
(i)



(ii)



(iii)



(iv)



Delete 24

(i)



(ii)



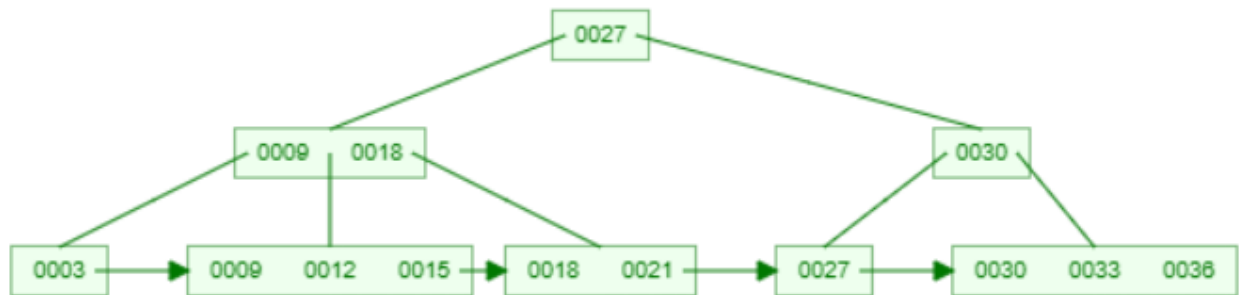
(iii)



(iv)



Final tree



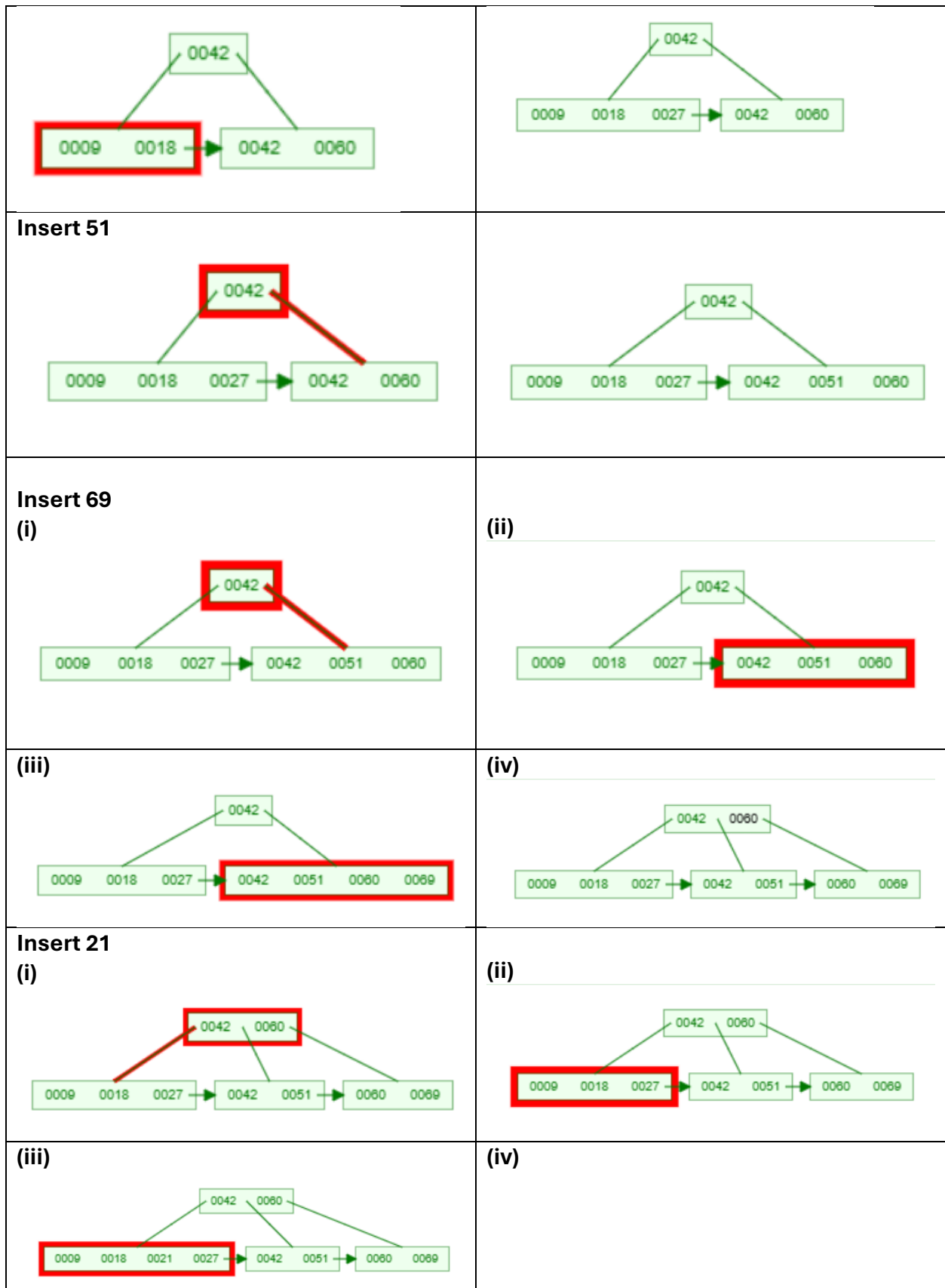
Question 5

Construct a B+ Tree of order 5 by inserting the keys 42, 18, 60, 9, 27, 51, 69, 21, 33, 57 in the given order. Indicate splitting of leaf and internal nodes, promoted keys, and linked leaves. After construction, insert 63 and 75, then delete 18 and 60. Show all redistribution or merging operations during deletion.

Constructing B+ tree

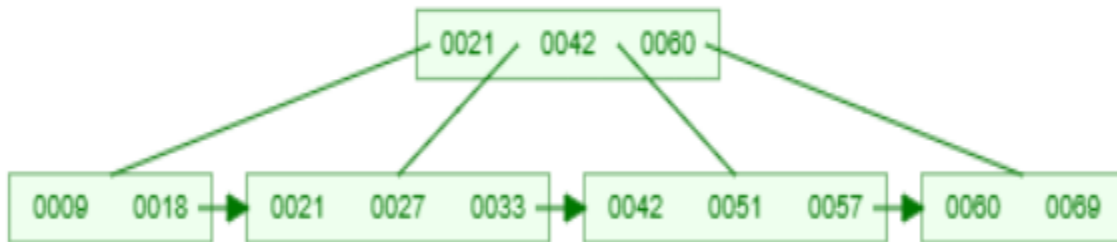
Insertion Operation {42, 18, 60, 9, 27, 51, 69, 21, 33, 57}

Insert 42 0042	
Insert 18 0018 0042	
Insert 60 0018 0042 0060	
Insert 9 0009 0018 0042 0060	
Insert 27	

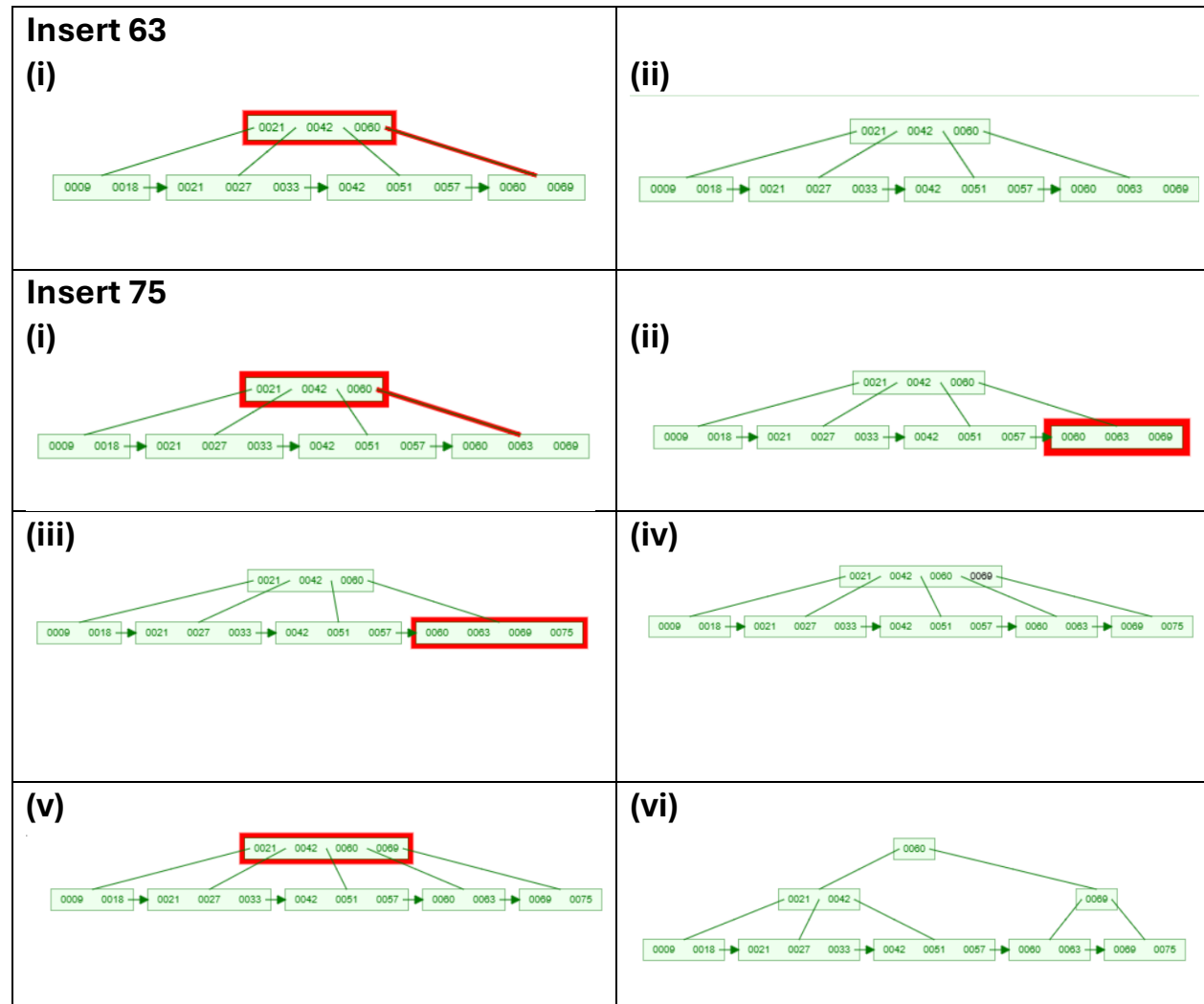


Insert 33 (i)	(ii)
(iii)	
Insert 57 (i)	(ii)
(iii)	

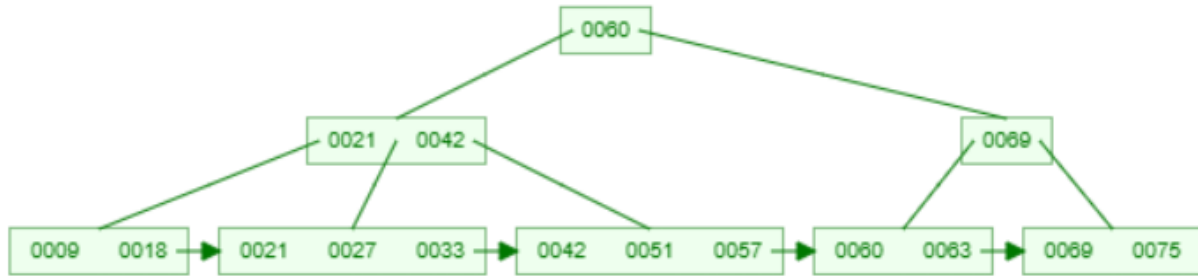
Finally tree



Insertion 63 and 75



Final tree after inserting 63 and 75



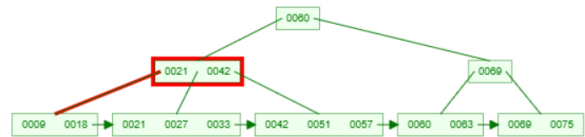
Deletion 18 and 60

Delete 18

(i)



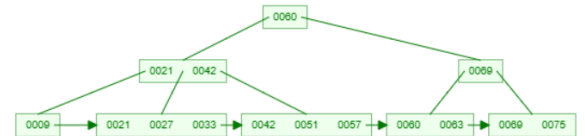
(ii)



(iii)



(iv)

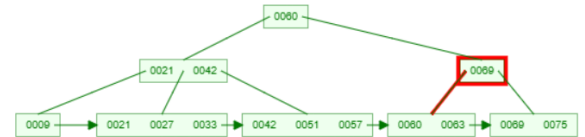


Delete 60

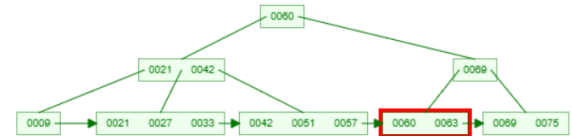
(i)



(ii)



(iii)



(iv)



Final tree

